Markov Models: Hidden Markov Models - The Three Fundamental Questions for HMMs - Probabilistic Context Free Grammars: Features of PCFGs - Questions for PCFGs - The Probability of a String.

Hidden Markov Model (HMM)

Hidden Markov Model (**HMM**) is a statistical Markov model in which the system being modeled is assumed to be a Markov process with unobserved (i.e. hidden) states.

In probability theory, a Markov model is a stochastic model used to model randomly changing systems. It is assumed that future states depend only on the current state, not on the events that occurred before it (that is, it assumes the Markov property).

Hidden Markov Models application include reinforcement learning and temporal pattern recognition such as speech, handwriting, gesture recognition, part-of-speech tagging, musical score following, partial discharges and bioinformatics.

The Three Fundamental Questions for HMMs

There are three fundamental questions that we want to know about an HMM:

1. Given a model **μ** = (A, B, **Π**), how do we **efficiently compute** how likely a certain observation is, that is P(O| **μ**)?

2. Given the observation sequence 0 and a model **μ** how do we c**hoose a state sequence** (X_1, . . . X_{T+1}) that best explains the observations?

3. Given an observation sequence 0, and a space of possible models found by varying the model parameters p = (A, B, **Π**), how do we **find the model** that best explains the observed data?

Lea is our imaginary friend and during the day she does either of these four things:

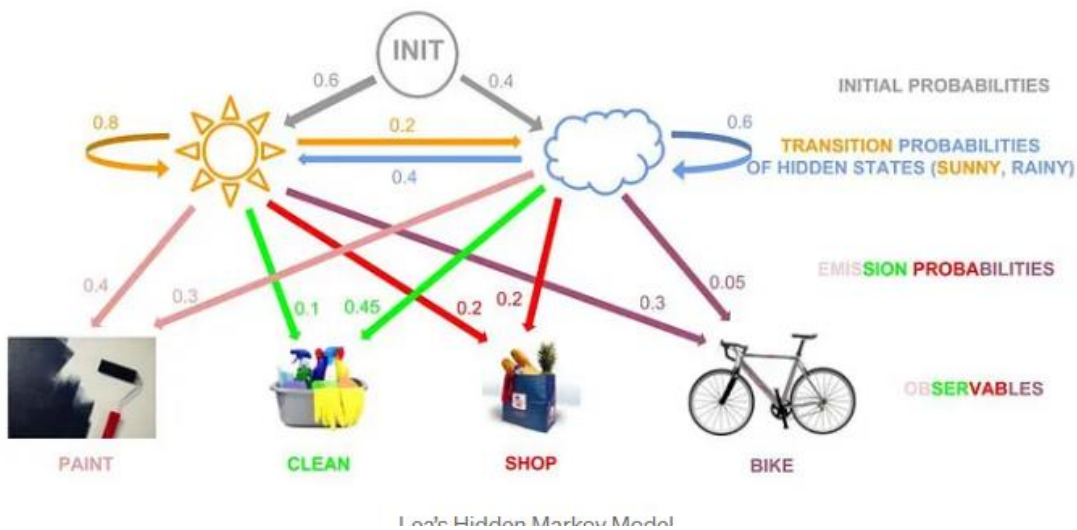
- Painting

- Cleaning the house
- Biking
- Shopping for groceries

But anyway, this is a list of what are called **observables**. Now, let's suppose that in four days Lea does the following: painting, cleaning, shopping, biking.

From this observation sequence we want to know whether the day has been sunny or rainy. These two are going to be our hidden states.

Here is a graph of our potential HMM:



Lea's Hidden Markov Model

a Hidden Markov Model consists of the following properties:

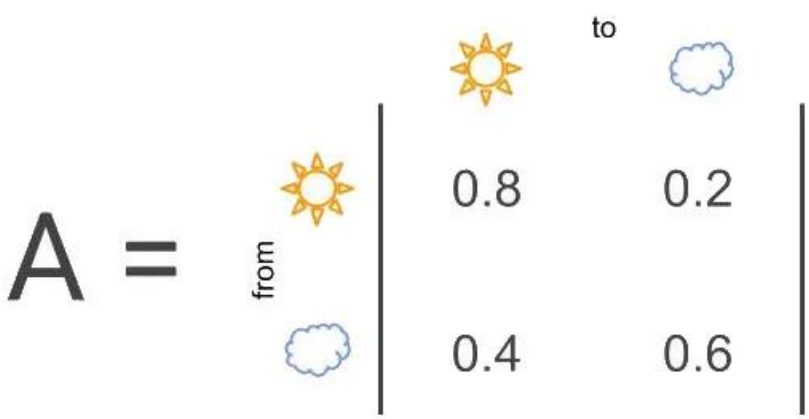
- **Hidden States *S***: in the example above the hidden states are Sunny and Rainy, and they get grouped into a set S.

- **Observables *O***: Paint, Clean, Shop and Bike. They get grouped into a set O.

- **Initial Probabilities π**: a matrix of the initial likelihood of the state at time $t = 0$. In this case the likelihood that it is Sunny on the first day is 0.6, while the likelihood that it is Rainy is 0.4. $\pi = |0.6, 0.4|$

  Note: every row of the following matrices must add up to 1 since they represent a probability.

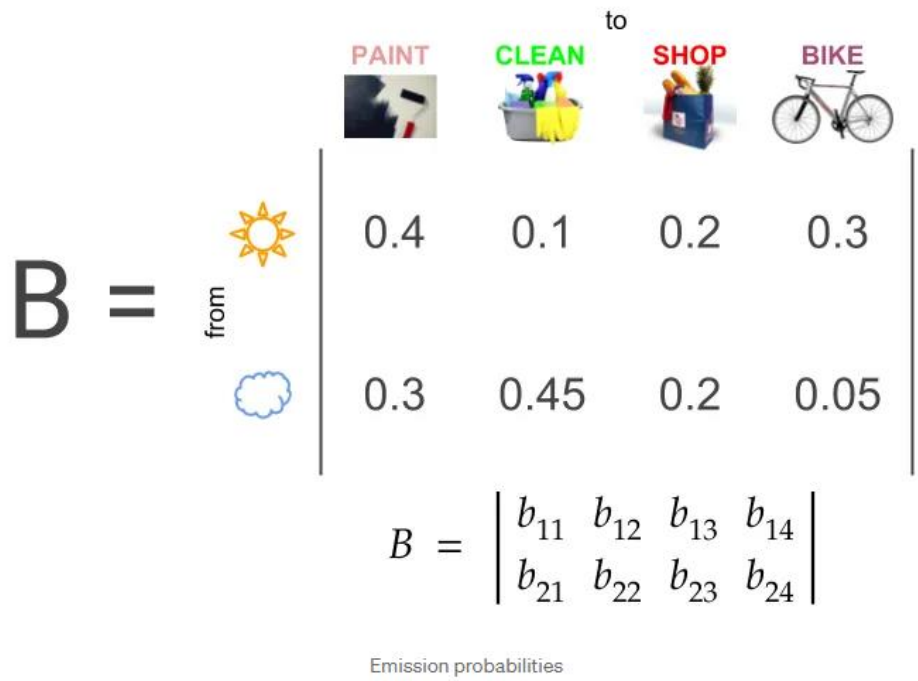
- **Transition Probabilities *A*:** a matrix that represents the probability of transitioning to another state given the current state. For example, if the current state is Sunny the probability that the day after is Sunny as well is 0.8, whereas the probability that the day after is Rainy is 0.2. Similarly if today is Rainy, the probability that tomorrow is Rainy as well is 0.6, while the probability that tomorrow is Sunny is 0.4.



$$A = \begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix}$$

Transition probabilities between states

- **Emission Probabilities *B:*** a matrix that represents the probability of seeing a specific observable given a hidden state. For example, the probability of Clean on a Sunny day is 0.1, whereas the probability of Clean on a Rainy day is 0.45.



Under more mathematical terms, we would describe this model's properties as such:

$$S = \{ S_{sunny}, S_{rainy} \} \quad (Hidden\ States)$$

$$O = \{ O_{clean}, O_{bike}, O_{shop}, O_{paint} \} \quad (Observables)$$

$$\pi = \begin{vmatrix} 0.6 & 0.4 \end{vmatrix} \quad (Initial\ Probabilities)$$

$$A = \begin{vmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{vmatrix} \quad (Transition\ Probabilities)$$

$$B = \begin{vmatrix} 0.4 & 0.1 & 0.2 & 0.3 \\ 0.3 & 0.45 & 0.2 & 0.05 \end{vmatrix} \quad (Emission\ Probabilities)$$

we're going to analyze the **first problem,** which asks the **likelihood of a certain observation sequence** deriving from the HMM model that we have initialized.

Let's take the initial example about Lea's activity in four days. The **observation sequence** is as follows: **Paint, Clean, Shop and Bike**.

$$O = \{ O_{paint}, O_{clean}, O_{shop}, O_{bike} \}$$

Observation sequence

So, what is the likelihood that this observation sequence *O* can derive from our HMM λ?

$$P(O|\lambda) = ???$$

There are two methods with which we can calculate this: the **Forward Algorithm and the Backward Algorithm.**

The Forward Algorithm

The Forward algorithm comprises of three steps:

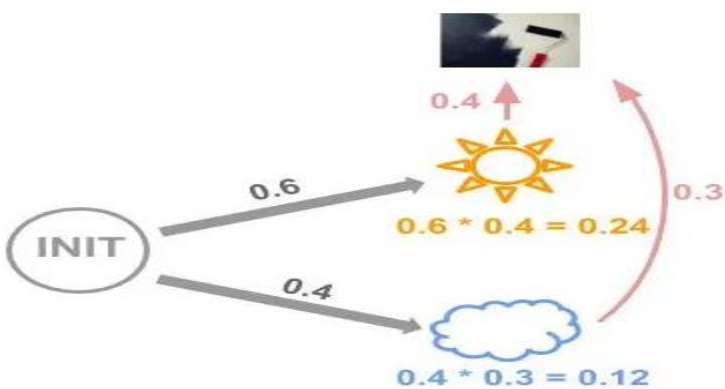
- Initialization
- Recursion
- Termination

Initialization

$$\alpha_1(i) = \pi_i \cdot b_i(O_1)$$

Forward algorithm initialization equation

The above equation means that the first forward variable is calculated by multiplying the initial probability of state ***i*** by the emission probability ***b*** of that state given the observable ***O*** at time **1.**



As can be seen the initial forward variable of the Sunny state is the initial probability of Sunny, 0.6, times the emission probability from Sunny to the observable Paint, 0.4. Hence, 0.24.

While the initial forward variable of the Rainy state is the initial probability of Rainy, 0.4, times the emission probability from Rainy to the observable Paint, 0.3. Hence, 0.12.

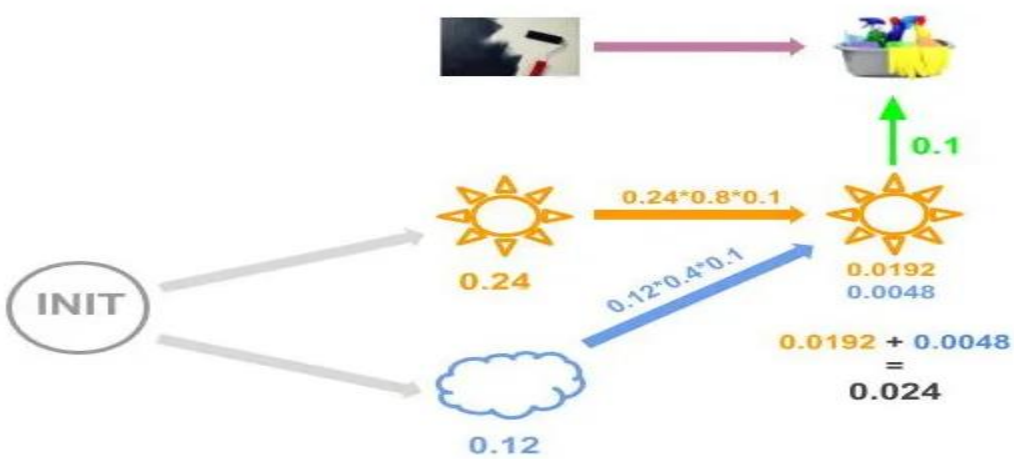
Recursion

$$\alpha_{t+1}(j) = \sum_{i=1}^{N} \alpha_t(i) \cdot a_{ij} \cdot b_j(O_{t+1})$$

Forward algorithm recursion equation

For *t = 1, 2, ..., T-1* we make use of the recursion equation which defines the forward variable of state *j* as the product of the previous forward variable of state *i,* multiplied by the transition probability *a* between the previous state *i* to state *j*, multiplied by the emission probability *b* from state *j* to the observable *O*.

let's have a look at the diagram below:



Here we are calculating the forward variable of state Sunny at time 2 by summing the results of two multiplications:

- The previous forward variable of the previous Sunny state, 0.24, times the transition probability from Sunny to Sunny, 0.8, times the emission

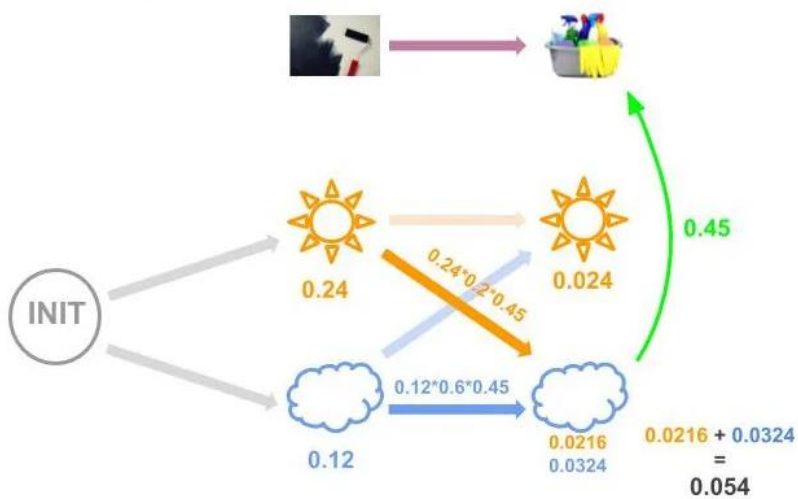
probability from Sunny to Clean, 0.1. **0.24 * 0.8 * 0.1 = 0.0192**

- The previous forward variable of the previous Rainy state, 0.12, times the transition probability from Rainy to Sunny, 0.4, times the emission probability from Sunny to Clean, 0.1. **0.12 * 0.4 * 0.1 = 0.0048**

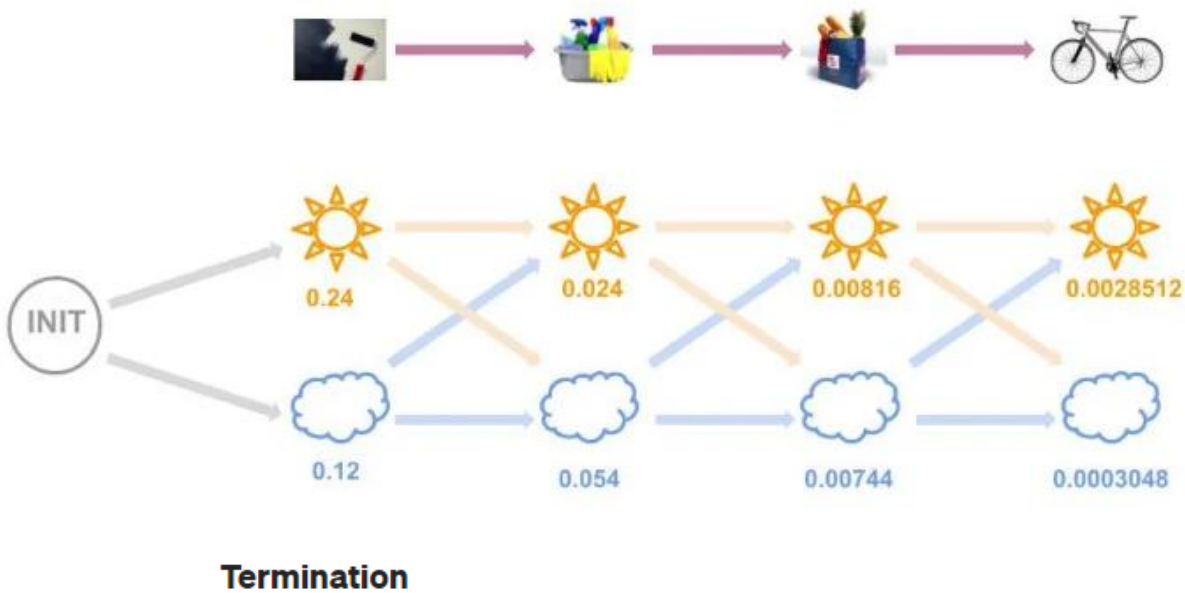
Then, according to the equation above, we sum these results and get our forward variable.

α = 0.0192 +0.0048 = 0.024

Similarly, for the next step we'll have a forward variable of 0.054 for the Rainy state:



and so on until we have all the forward variables:



$$P(O|\lambda) = \sum_{i=1}^{N} \alpha_T(i)$$

Termination

This final equation tells us that to find the probability of an observation sequence O deriving from an HMM model λ, we need to sum up all the forward variables at time T, i.e. all the variables of every state at the end of the observation sequence. Hence, in our example above,

P(O|λ) = 0.0028512 + 0.0003048 = 0.003156

<u>The Backward Algorithm</u>

Usually, to find the solution to the Likelihood problem we don't really require to know the Backward Algorithm. However, its explanation and resolution is a good litmus test to show that the Forward algorithm works proficiently, and moreover, by understanding it now, we can be ready for when the time will come to use it for the resolution of the third problem of Learning.

The Backward Algorithm is similar to the Forward algorithm, but like the name suggests it goes backward in time. There's again an Initialization, a Recursion and a Termination.

Initialization

$$\beta_T(i) = 1$$

Initialization

What this simple equation is telling us is that at time T (at the end of the observation sequence) the backward variables of every state is equal to 1.

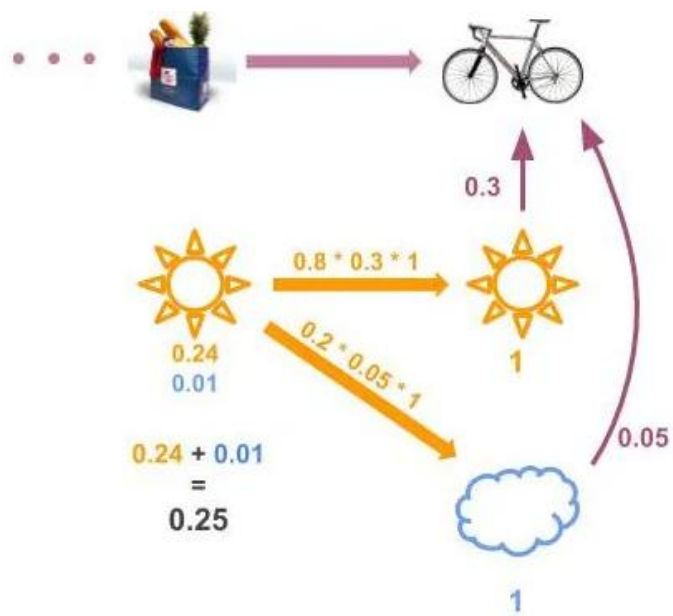
Initialization of Backward Algorithm

Recursion

$$\beta_t(i) = \sum_{j=1}^{N} a_{ij} \cdot b_j(O_{t+1}) \cdot \beta_{t+1}(j)$$

Recursion

The equation above tells us to sum up all the multiplications of the transition probability from the current State ***i*** to the next State ***j*,** times the emission probability of the observable ***O*** at time ***t+1*** from the next State ***j***, times the backward variable of the next State ***j*** at time ***t+1***.



Recursion of Backward Algorithm 1

We’re going back in time. At time T where the final observable in the observation sequence was Bike, the backward variables have been initialized as equal to 1.

By going backwards we’re going to calculate the backward variable of the previous Sunny state. This consists of summing up two multiplications:

- Transition probability from Sunny to Sunny, 0.8, times the emission probability of Bike deriving from the Sunny state at time *t+1*, 0.3, times the backward variable of the Sunny State at time *t+1*, 1.

  0.8 * 0.3 * 1 = 0.24

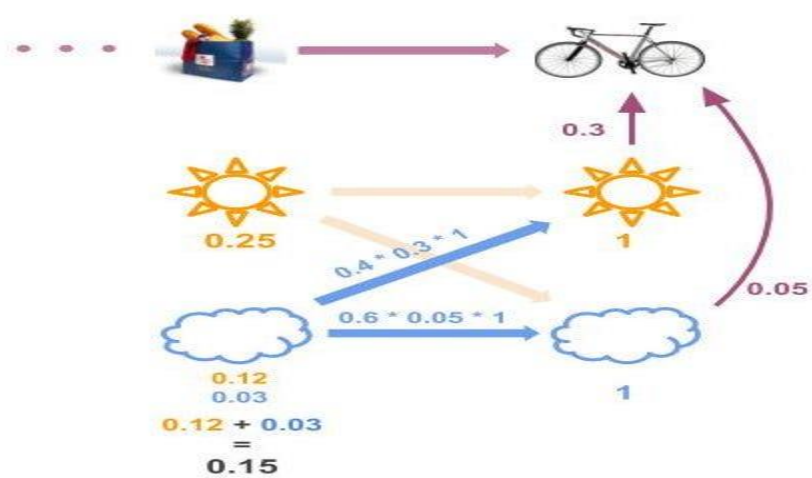
- Transition probability from Sunny to Rainy, 0.2, times the emission probability of Bike deriving from the Rainy state at time *t+1*, 0.05, times the backward variable of the Sunny State at time *t+1*, 1.

  0.2 * 0.05 * 1 = 0.01

Then, according to the equation above, we sum these results and get our backward variable.

β = 0.24 +0.01 = 0.25

Similarly for the Rainy state:



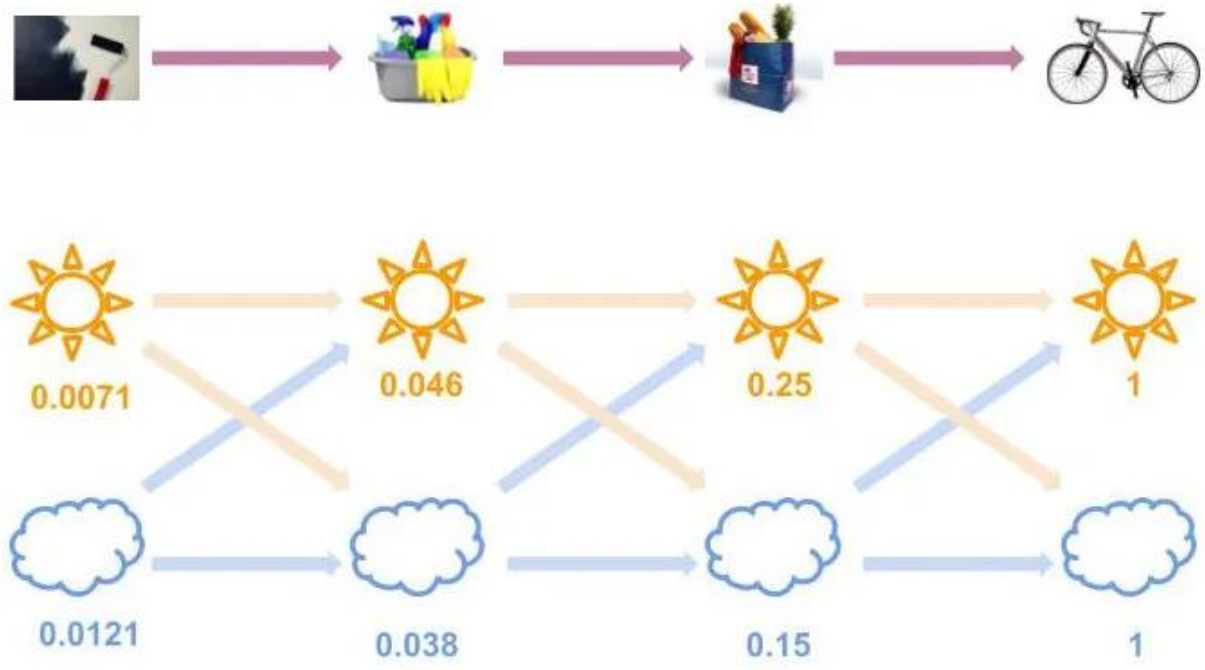
Like before, this is what we’re going to sum up:

- Transition probability from Rainy to Sunny, 0.4, times the emission probability from the Sunny state to Bike at time *t+1*, 0.3, times the backward variable of the Sunny State at time *t+1*, 1. **0.4 * 0.3 * 1 = 0.12**

- Transition probability from Rainy to Rainy, 0.6, times the emission probability from the Rainy state to Bike at time *t+1*, 0.05, times the backward variable of the Sunny State at time *t+1*, 1. **0.6 * 0.05 * 1 = 0.03**

β = 0.12 +0.03 = 0.15

And so on until we have all the backward variables:



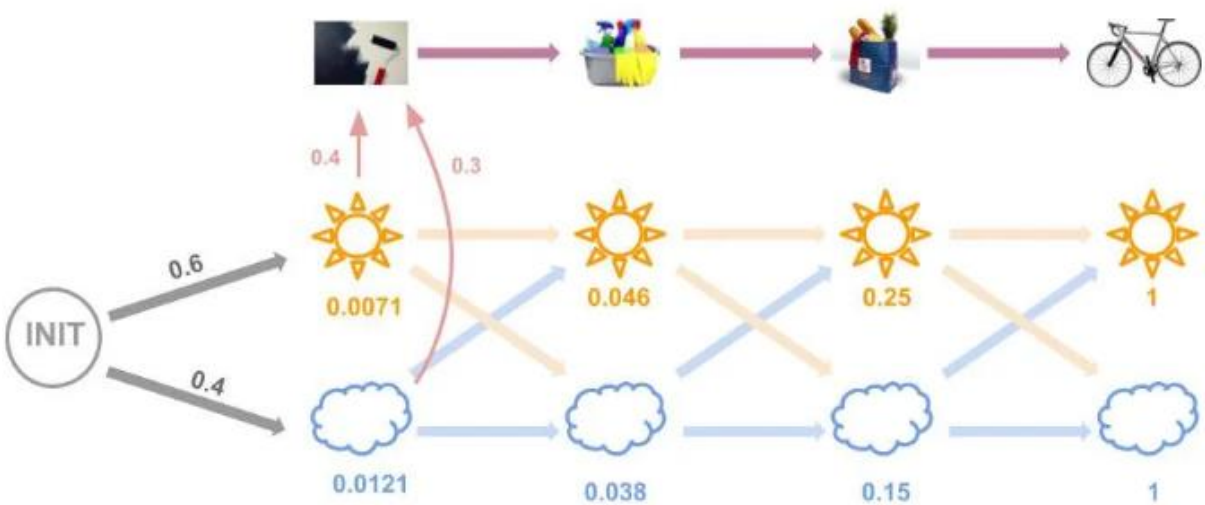
Termination

$$P(O|\lambda) = \sum_{i=1}^{N} \pi_i \cdot b_i(O_1) \cdot \beta_1(i)$$

Termination

What this equation wants us to do is to sum the multiplications of the initial probability ***π*** of state ***i***, times the emission probability ***b*** of the observable ***O*** at time ***t=1*** from state ***i*,** times the backward variable at time ***t=1*** of state ***i*.**

Let's clarify this in the diagram below:



Termination of Backward Algorithm

As we can see, I have highlighted the probabilities we're going to need to calculate our final probability.

As per equation we're going to sum the following multiplications:

- The initial probability ***π*** of the Sunny state, 0.6, times the emission probability from Sunny state at time *t=1* to the observable Paint, 0.4, times

the backward variable of the Sunny state at time *t=1*, 0.0071. **0.6 * 0.4 * 0.0071 = 0.001704**

- The initial probability $\boldsymbol{\pi}$ of the Rainy state, 0.4, times the emission probability from Rainy state at time *t=1* to the observable Paint, 0.3, times the backward variable of the Rainy state at time *t=1*, 0.0121. **0.4 * 0.3 * 0.0121 = 0.001452**

P(O|λ) = 0.001704 + 0.001452 = 0.003156

And as we can see, the result is the same as that of the Forward Algorithm.

Viterbi Algorithm

Finding the best state sequence

The Viterbi Algorithm is a dynamic programming solution for finding the **most probable hidden state sequence**.

The goal is to find the **single** state sequence $S=(s_1, s_2,\ldots, s_N)$ that "best explains" a given observation sequence $O=(O_1, O_2,\ldots, O_N)$.

If we have a set of states **Q** and a set of observations **O**, we are trying to find the state sequence that maximizes **P(Q|O).**

By conditional probability, we can transform **P(Q|O) to P(Q,O)/P(O),** but there is no need in finding P(O) as P(O) does not pertain to changes in state sequences.

$$
\begin{aligned}
P(Q,O) &= P(O|Q)P(Q) \\
&= P(o_1 \dots o_T | q_1 \dots q_T) \prod_{i=1}^{T} P(q_i | q_1 \dots q_{i-1}) \\
&= P(o_1 \dots o_T | q_1 \dots q_T) \prod_{i=1}^{T} P(q_i | q_{i-1}) \\
&= \prod_{i=1}^{T} P(o_i | o_1 \dots o_{i-1}, q_1 \dots q_{i-1}) \prod_{i=1}^{T} P(q_i | q_{i-1}) \\
&= \prod_{i=1}^{T} P(o_i | q_i) P(q_i | q_{i-1})
\end{aligned}
$$

T is the number of observations in the sequence.

So, if we wanted to find the state sequence to maximize P(Q, O), you could imagine this would be quite expensive as we would have to maximize P(Q, O) for all possible state sequences.

This is where **the Viterbi algorithm comes in**. The Viterbi algorithm is an iterative approach to solving for the most likely sequence. Rather than finding the most probable hidden state sequence for all the observations, we just want to find the next most probable hidden state. We iteratively find the most probable states until we are done with the sequence of observations.

Viterbi Algorithm

There are some symbols we should denote for the Viterbi algorithm.

$\delta_{q_i}(t)$ = the probability of taking the maximal path up to time $t-1$ ending in state q_i at time t for observations $o_1 \ldots o_t$

$\psi_{q_i}(t)$ = The sequence resulting in the most maximal probability up to state q_i at time t

The Viterbi Algorithm is composed of **three** steps.

We first create a start state q*. We then find the probabilities of the initial states and the observations given the initial states. In this case, $P(q_i|q^*)$ is the probability that the start state is $q_i|$.

$$\delta_{q_i}(1) = P(q_i|q^*)P(o_i|q_i)$$

Step 2: Induction

We perform the induction step when **t** is greater than or equal to 2 and less than or equal to **T** where **T** is the number of observations + 1 (the plus 1 comes from the added start state). T represents to total number of observations.

$$\delta_{q_i}(t) = \max_{q_j} P(q_i | q_j) P(o_i | q_i) \delta_{q_j}(t-1)$$

$$\psi_{q_i}(t) = \arg\max_{q_j} P(q_i | q_j) P(o_i | q_i) \delta_{q_j}(t-1)$$

Step 3: Termination

$$P(Q, O) = \max_{q_j} \delta_{q_j}(T)$$

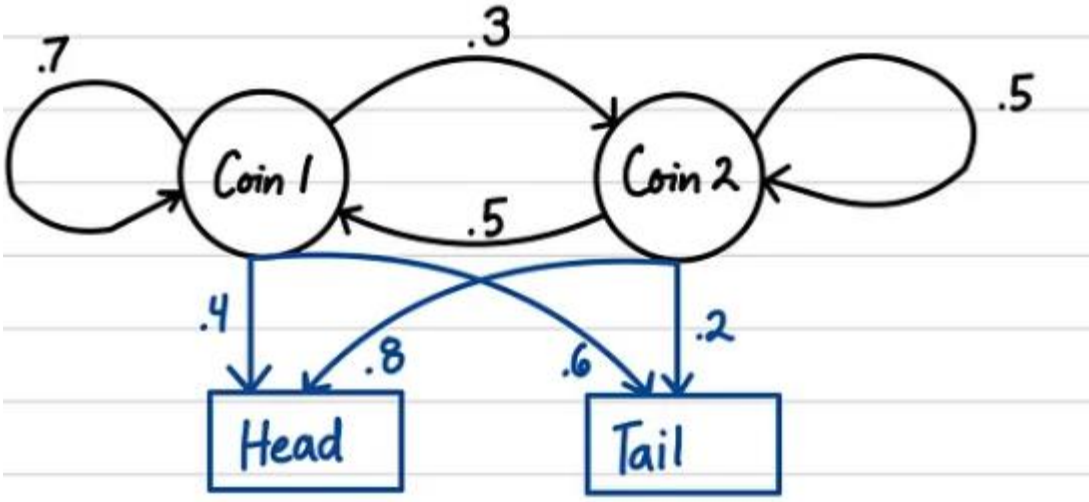
Best last state:

$$\hat{q}_T = \arg\max_{q_j} \delta_{q_j}(T)$$

Getting best states by backtracking

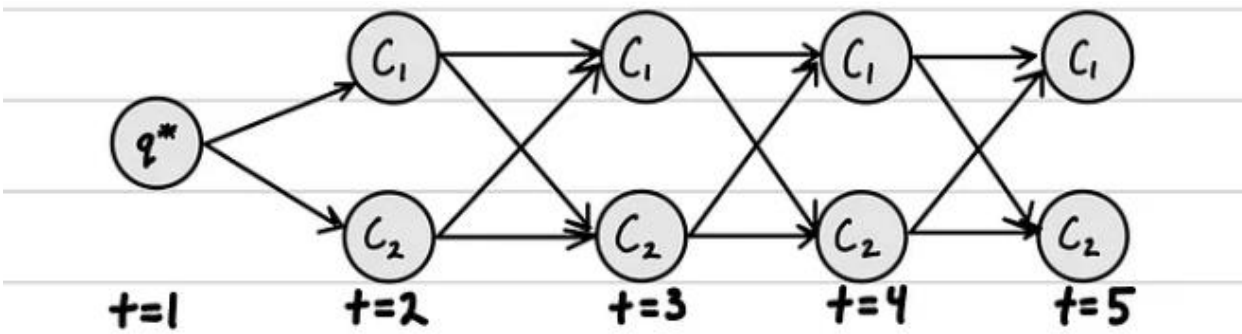
$$\hat{q}_t = \psi_{\hat{q}_{t+1}}(t) \quad \text{where } t = k-1, \ldots, 1$$

Example



So, let's go ahead and start finding the most probable hidden state sequence when we observed head, tail, tail, and head.

I like to visualize the Viterbi algorithm using a trellis.



we need to first initialize,

Initialization Step

$$\delta_{q_i}(1) = P(q_i \mid q^*) P(o_i \mid q_i)$$

$$\delta_{C_1}(1) = P(C_1 \mid q^*) P(H \mid C_1) = \pi_{C_1} P(H \mid C_1) = .6 \cdot .4 = 0.24$$

$$\delta_{C_2}(1) = P(C_2 \mid q^*) P(H \mid C_2) = \pi_{C_2} P(H \mid C_2) = .4 \cdot .8 = .32$$

Here we are calculating the probability of observing head at coin 1 and head at coin 2 at time step 1.

Instead of user P(qi | q*), we use the initial state distribution.

Induction Steps at time t=2

$$\delta_{q_i}(2) = \max_{q_j} P(q_i \mid q_j) P(o_i \mid q_i) \delta_{q_j}(1)$$

$$\delta_{c_1}(2) = \max_{q_j} P(c_1 \mid q_j) P(T \mid c_1) \delta_{q_j}(1)$$

$$= \max\Big(P(c_1 \mid c_1) P(T \mid c_1) \delta_{c_1}(1), \; P(c_1 \mid c_2) P(T \mid c_1) \delta_{c_2}(1) \Big)$$

$$= \max\Big(0.7 \cdot 0.6 \cdot 0.24, \rightarrow \max \; 0.5 \cdot 0.6 \cdot 0.32 \Big) = 0.1008$$

$$\psi_{c_1}(2) = \arg\max_{q_j} P(c_1 \mid q_j) P(T \mid c_1) \delta_{q_j}(1)$$

$$= c_1$$

$$\delta_{c_2}(2) = \max_{q_j} P(c_2 | q_j) P(T | c_2) \delta_{q_j}(1)$$

$$= \max\Big(P(c_2 | c_1) P(T | c_2) \delta_{c_1}(1), \; P(c_2 | c_2) P(T | c_2) \delta_{c_2}(1) \Big)$$

$$= \max\Big(0.3 \cdot 0.2 \cdot 0.24, \; 0.5 \cdot 0.2 \cdot 0.32 \rightarrow \max \Big) = 0.0032$$

$$\psi_{c_2}(2) = \arg\max_{q_j} P(c_2 | q_j) P(T | c_2) \delta_{q_j}(1)$$

$$= c_2$$

$$\delta_{c_1}(3) = \max_{q_j} P(c_1 | q_j) P(T | c_1) \delta_{q_j}(2)$$

$$= \max(0.7 \cdot 0.6 \cdot 0.1008, 0.5 \cdot 0.6 \cdot 0.0032)$$

$$= 0.04236$$

$$\delta_{c_2}(3) = \max_{q_j} P(c_2 | q_j) P(T | c_2) \delta_{q_j}(2)$$

$$= \max(0.3 \cdot 0.2 \cdot 0.1008, 0.5 \cdot 0.2 \cdot 0.0032)$$

$$= 0.006048$$

$$\delta_{c_1}(4) = \max_{q_j} P(c_1 | q_j) P(H | c_1) \delta_{q_j}(3)$$

$$= \max(0.7 \cdot 0.4 \cdot 0.04236, 0.5 \cdot 0.4 \cdot 0.006048)$$

$$= 0.0118608$$

$$\delta_{c_2}(4) = \max_{q_j} P(c_2 | q_j) P(H | c_2) \delta_{q_j}(3)$$

$$= \max(0.3 \cdot 0.8 \cdot 0.04236, 0.5 \cdot 0.8 \cdot 0.006048)$$

$$= 0.0101664$$

$$\delta_{c_1}(5) = \max_{q_j} P(c_1 | q_j) P(H | c_1) \delta_{q_j}(4)$$

$$= \max(0.7 \cdot 0.4 \cdot 0.04236,\ 0.5 \cdot 0.4 \cdot 0.006048)$$

$$= 0.0118608$$

$$\delta_{c_2}(5) = \max_{q_j} P(c_2 | q_j) P(H | c_2) \delta_{q_j}(4)$$

$$= \max(0.3 \cdot 0.8 \cdot 0.04236,\ 0.5 \cdot 0.8 \cdot 0.006048)$$

$$= 0.0101664$$

The state sequence that best explains the observations.

There is more than one way to think about doing this. One way to proceed would be to choose the states individually.

The Three Fundamental Questions for HMMs

(1)The Evaluation Problem

Given an HMM λ and a sequence of observations $O = o_1, o_2, \ldots, o_T$, what is the probability that the observations are generated by the model, $p\{O|\lambda\}$?

(2)The Decoding Problem

Given a model λ and a sequence of observations $O = o_1, o_2, \ldots, o_T$, what is the most likely state sequence in the model that produced the observations?

(3)The Learning Problem

Given a model λ and a sequence of observations $O = o_1, o_2, \ldots, o_T$, how should we adjust the model parameters $\{A, B, \pi\}$ in order to maximize $p\{O|\lambda\}$.

Evaluation problem can be used for isolated (word) recognition. Decoding problem is related to the continuous recognition as well as to the segmentation. Learning problem must be solved, if we want to train an HMM for the subsequent use of recognition tasks.

Probabilistic Context Free Grammars: Features of PCFGs

A Context Free Grammars consists of A set of terminals $\{ W^k \}$, k = 1,... V .

A set of nonterminals $\{ N^i \}$, i = 1,... n .

A designated start symbol N^1 A set of rules $\{ N^1 \rightarrow \xi^j \}$ ξ^j where is a sequence of terminals and nonterminals.

A **probabilistic context free grammar** is a context free grammar with probabilities attached to the rules.

Model Parameters

The model parameters are the probabilities assigned to grammar rules.

Computing Probabilities

How the model assigns probabilities to strings and to analyses of strings.

Exploiting Probabilities in Parsing

How to find the most probable parse given the model.

Estimating Parameters

How rule probabilities are estimated from a syntactically annotated corpus.

Rule Probabilities

Probability of an expansion given the category being expanded. P(γ | A), the probability that the sequence of grammar symbols γ will expand category A. Given the definition of a conditional probability, this means the probabilities of all the expansions of a single category must add to 1.

For example:

S -> NP VP, .8 | Aux NP VP, .2

Given that S is being expanded, the probability of the NP VP rule is .8; the probability of the Aux NP VP expansion is .2. The rule probabilities are the parameters of a PCFG model.

Tree Probability

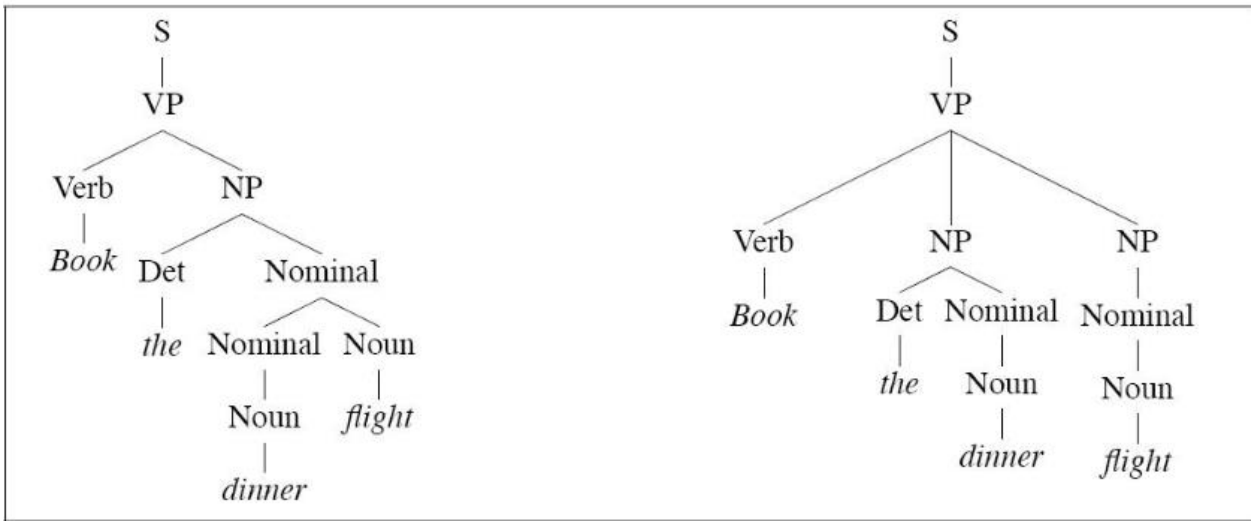
The probability of a tree given all the rules used to construct it. This will be the product of the probabilities of all the rules used to construct it. We motivate this below.

For now an example. Given this grammar:

Grammar		Lexicon
S → *NP VP*	[.80]	*Det* → *that* [.10] | *a* [.30] | *the* [.60]
S → *Aux NP VP*	[.15]	*Noun* → *book* [.10] | *flight* [.30]
S → *VP*	[.05]	| *meal* [.15] | *money* [.05]
NP → *Pronoun*	[.35]	| *flights* [.40] | *dinner* [.10]
NP → *Proper-Noun*	[.30]	*Verb* → *book* [.30] | *include* [.30]
NP → *Det Nominal*	[.20]	| *prefer*; [.40]
NP → *Nominal*	[.15]	*Pronoun* → *I* [.40] | *she* [.05]
Nominal → *Noun*	[.75]	| *me* [.15] | *you* [.40]
Nominal → *Nominal Noun*	[.20]	*Proper-Noun* → *Houston* [.60]
Nominal → *Nominal PP*	[.05]	| *NWA* [.40]
VP → *Verb*	[.35]	*Aux* → *does* [.60] | *can* [40]
VP → *Verb NP*	[.20]	*Preposition* → *from* [.30] | *to* [.30]
VP → *Verb NP PP*	[.10]	| *on* [.20] | *near* [.15]
VP → *Verb PP*	[.15]	| *through* [.05]
VP → *Verb NP NP*	[.05]	
VP → *VP PP*	[.15]	
PP → *Preposition NP*	[1.0]	

the sentence

Book the dinner flight



Rules			P	Rules			P
S	→	VP	.05	S	→	VP	.05
VP	→	Verb NP	.20	VP	→	Verb NP NP	.10
NP	→	Det Nominal	.20	NP	→	Det Nominal	.20
Nominal	→	Nominal Noun	.20	NP	→	Nominal	.15
Nominal	→	Noun	.75	Nominal	→	Noun	.75
				Nominal	→	Noun	.75
Verb	→	book	.30	Verb	→	book	.30
Det	→	the	.60	Det	→	the	.60
Noun	→	dinner	.10	Noun	→	dinner	.10
Noun	→	flights	.40	Noun	→	flights	.40

$P(T_{left}) = .05 * .20 * .20 * .20 * .75 * .30 * .60 * .10 * .40 = \mathbf{2.2 \times 10^{-6}}$

$P(T_{right}) = .05 * .10 * .20 * .15 * .75 * .75 * .30 * .60 * .10 * .40 = \mathbf{6.1 \times 10^{-7}}$

Therefore the more probable of the two parses is Tleft, which is the intuitive result.

String Probability

Sum of the tree probabilities for all analyses of the string.

$P(W_{14}) = P(T_{left}) + P(T_{right}) = \mathbf{2.2 \times 10^{-6}} + \mathbf{6.1 \times 10^{-7}}$

Our primary subject of interest is finding the highest probability parse for a sentence.